

操作系统, 2026

第6讲 进程间通信(2/2)

清华大学电子工程系
马洪兵

OPERATING SYSTEM

大纲

01

Linux的互斥和同步机制

02

Windows的互斥和同步机制

03

进程间高级通信

Linux的互斥和同步机制

- Linux应用程序采用POSIX标准用于进程/线程的同步和互斥
 - POSIX同步和互斥API包括互斥锁、条件变量和读写锁
 - POSIX支持信号量（SEM扩展）
- Linux也支持System V信号量

互斥锁(mutex)

- 互斥锁(mutex)用于保护临界区代码，即进入临界区时获取锁，退出临界区释放锁
- 主要的相关函数
 - `int pthread_mutex_init(pthread_mutex_t *mutex, const pthread_mutexattr_t *mutexattr)`
 - `int pthread_mutex_lock(pthread_mutex *mutex)`
 - `int pthread_mutex_unlock(pthread_mutex *mutex)`
 - `int pthread_mutex_destroy(pthread_mutex *mutex)`

互斥锁(mutex)

■ 初始化

- 调用 `pthread_mutex_init()`
- 或者静态赋值 `pthread_mutex_t mutex=PTHREAD_MUTEX_INITIALIZER`

■ 加锁:

- `pthread_mutex_lock`——阻塞等待锁

■ 解锁

- `pthread_mutex_unlock()`

■ 清除锁

- `pthread_mutex_destroy()`——此时锁必需unlock, 否则返回EBUSY

条件变量(cond)

- 互斥锁能够解决临界资源的互斥访问问题，但是有些问题并不能用互斥解决，例如线程T1和T2都要访问全局变量i、j

T1:

```
...  
pthread_mutex_lock();  
i++;  
j--;  
pthread_mutex_unlock();  
...
```

T2:

```
...  
pthread_mutex_lock();  
if(i == j) {  
    do_something();  
}  
pthread_mutex_unlock();  
...
```

- 假设初值i=3, j=7, 执行顺序为T1抢到锁 → T2抢到锁 → T1抢到锁 → T1抢到锁 → T2抢到锁, 导致do_something()永远不会执行
- 此外, T2只在一种情况下执行do_something(), 不停地申请和释放锁造成资源浪费

条件变量(cond)

- 与互斥锁不同，条件变量是用来等待而不是用来上锁的。条件变量用来自动阻塞一个线程，直到某特殊情况发生为止
- 主要包括两个操作：一个线程等待条件变量的条件成立而阻塞；另一个线程使条件成立(给出条件成立信号)

条件变量(cond)

■主要的相关函数

- `int pthread_cond_init(pthread_cond_t *cond, pthread_condattr_t *cond_attr)`
- `int pthread_cond_wait(pthread_cond_t *cond, pthread_cond_t *cond)`
- `int pthread_cond_signal(pthread_cond_t *cond)`
- `int pthread_cond_destroy(pthread_cond_t *cond)`

条件变量(cond)

T1:

```
...  
pthread_mutex_lock();  
if(i == j) {  
    pthread_mutex_unlock();  
    pthread_cond_signal();  
}  
i++;  
j--;  
pthread_mutex_unlock();  
...
```

T2:

```
...  
pthread_mutex_lock();  
if(i != j) {  
    pthread_mutex_unlock();  
    pthread_cond_wait();  
}  
do_something();  
pthread_mutex_unlock();  
...
```

读写锁(rwlock)

- 在对数据的读写应用中，多数情况下是大量的读操作和较少的写操作，针对这一需要POSIX提供了读写锁
- 读写锁有三种状态：读模式加锁、写模式加锁、不加锁。一次只有一个线程可以占有写模式下的读写锁，但是可以有多个线程同时占有读模式下的读写锁
- 当读写锁处于写模式加锁状态时，在它解锁之前，任何试图对这个读写锁加锁的线程都会被阻塞
- 当读写锁处于读模式加锁状态时，所有试图对这个读写锁以读模式加锁的线程都会获得访问权；如果有线程试图以写模式加锁，该线程必须阻塞直到所有的线程释放锁，但是读写锁会阻塞随后的读模式加锁请求，从而避免读锁长期占用写锁得不到满足的情况

读写锁(rwlock)

■主要的相关函数

- `int pthread_rwlock_init(pthread_rwlock_t *rwlock, pthread_rwlockattr_t *rwlock_attr)`
- `int pthread_rwlock_rdlock(pthread_rwlock_t *rwlock)`
- `int pthread_rwlock_wrlock(pthread_rwlock_t *rwlock)`
- `int pthread_rwlock_unlock(pthread_rwlock_t *rwlock)`
- `int pthread_rwlock_destroy(pthread_rwlock_t *rwlock)`

信号量

■ POSIX提供两种信号量

1. 有名(named)信号量
2. 基于内存的(memory-based)信号量, 也称为无名(unnamed)信号量

信号量——无名信号量

- 无名信号量的创建就像声明一般的变量一样简单，例如：

```
sem_t sem_id;
```

- 常见的无名信号量相关函数

- `int sem_init(sem_t *sem, int pshared, unsigned int value)`

- `int sem_getvalue(sem_t *sem, int *sval)`

- `int sem_wait(sem_t *sem)`

- `int sem_post(sem_t *sem)`

- `int sem_destroy(sem_t *sem)`

信号量——有名信号量

- 有名信号量的特点是把信号量的值保存在文件中。这决定了它的用途非常广：既可以用于线程，也可以用于相关进程间，甚至是不相关进程
- 有名信号量在使用的时候，和无名信号量共享sem_wait和sem_post函数；两者的区别是有名信号量使用sem_open代替sem_init，另外在结束的时候要像关闭文件一样去关闭这个有名信号量

信号量

有名信号量

无名信号量

`sem_open()`

`sem_init()`

`sem_wait()`
`sem_post()`

`sem_close()`

`sem_destroy()`

生产者-消费者问题

- sem_t full; /*满信号量*/
- sem_t empty; /*空信号量*/
- pthread_mutex_t mutex; /*互斥量: 保护缓冲区互斥访问*/

Producer

P(empty);
P(mutex); //进入区
one unit --> buffer;
V(mutex);
V(full); //退出区

Consumer

P(full);
P(mutex); //进入区
one unit <-- buffer;
V(mutex);
V(empty); //退出区

生产者-消费者问题

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <pthread.h>
#include <semaphore.h>

#define BUFFSIZE 4
#define N_PRODUCER 6
#define N_CONSUMER 6

int buffer[BUFFSIZE];
int in = 0;           //生产产品位置
int out = 0;          //消费产品位置

sem_t full;           //满信号量
sem_t empty;          //空信号量
pthread_mutex_t mutex; //互斥量: 保护缓冲区互斥访问
```

```
void *producer(void *args)
{
    int id = *(int *)args;
    int product_id = rand() % 10000;
    printf("Producer %d started\n", id);
    sem_wait(&empty);           //获取empty信号量
    pthread_mutex_lock(&mutex); //获取互斥量

    printf("Producer #%d is producing product: %d\n", id, product_id);
    buffer[in] = product_id;
    in = (in + 1) % BUFFSIZE;

    pthread_mutex_unlock(&mutex); //释放互斥量
    sem_post(&full);              //释放full信号量

    sleep(5);
    printf("Producer %d ended\n", id);
    return 0;
}
```

生产者-消费者问题

```
void *consumer(void *args)
{
    int id = *(int *)args;
    int product_id;
    printf("Consumer %d started\n", id);
    sem_wait(&full);                //获取full信号量
    pthread_mutex_lock(&mutex);      //获取互斥量
    product_id = buffer[out];
    out = (out + 1) % BUFSIZE;
    printf("Consumer #%d is consuming product: %d\n", id, product_id);

    pthread_mutex_unlock(&mutex);    //解锁互斥量
    sem_post(&empty);                //释放empty信号量

    sleep(5);
    printf("Consumer %d ended\n", id);
    return 0;
}
```

```
int main()
{
    int i;
    pthread_t thread[N_PRODUCER + N_CONSUMER];

    sem_init(&full, 0, 0);
    sem_init(&empty, 0, BUFSIZE);
    pthread_mutex_init(&mutex, NULL);

    //创建生产者
    for (i = 0; i < N_PRODUCER; ++i)
        pthread_create(&thread[i], NULL, producer, (void *)&i);
    //创建消费者
    for (i = 0; i < N_CONSUMER; ++i)
        pthread_create(&thread[N_CONSUMER + i], NULL, consumer, (void *)&i);

    //等待线程结束
    for (i = 0; i < N_PRODUCER + N_CONSUMER; ++i)
        pthread_join(thread[i], NULL);

    pthread_mutex_destroy(&mutex);
    sem_destroy(&empty);
    sem_destroy(&full);
    return 0;
}
```

大纲

01

Linux的互斥和同步机制

02

Windows的互斥和同步机制

03

进程间高级通信

Windows的互斥和同步机制

- Windows内核支持三种同步对象
 - 互斥对象(Mutex)
 - 信号量对象(Semaphore)
 - 事件对象(Event)

Mutex对象

- 互斥对象，相当于互斥信号量，在一个时刻只能被一个线程使用
- 有关的API：
 - CreateMutex创建或打开一个互斥对象，返回对象句柄
 - OpenMutex返回一个已存在的互斥对象的句柄，用于后续访问
 - ReleaseMutex释放对互斥对象的占用，使之成为可用

Mutex对象

■ CreateMutex

```
HANDLE CreateMutex(  
    LPSECURITY_ATTRIBUTES lpMutexAttributes, // SD  
    BOOL bInitialOwner, // initial owner  
    LPCTSTR lpName // object name  
);
```

- 对象名称是由用户给出的字符串。不同进程中用同样的名称来创建或打开对象，从而获得该对象在本进程的句柄

Semaphore对象

- 相当于资源信号量，取值在0到指定最大值之间，用于限制并发访问的线程数
- 有关的API：
 - CreateSemaphore创建一个信号量对象，指定最大值和初值，返回对象句柄
 - OpenSemaphore返回一个已存在的信号量对象的句柄，用于后续访问
 - ReleaseSemaphore释放对信号量对象的占用

Semaphore对象

■ CreateSemaphore

```
HANDLE CreateSemaphore(  
    LPSECURITY_ATTRIBUTES lpSemaphoreAttributes, // SD  
    LONG lInitialCount, // initial count  
    LONG lMaximumCount, // maximum count  
    LPCTSTR lpName // object name  
);
```


Event对象

- 事件对象，相当于“触发器”，可通知一个或多个线程某事件的出现
- 有关的API：
 - CreateEvent创建一个事件对象，返回对象句柄；
 - OpenEvent返回一个已存在的事件对象的句柄，用于后续访问；
 - SetEvent和PulseEvent设置指定事件对象为有信号状态；
 - ResetEvent设置指定事件对象为无信号状态(nonsignaled)

同步对象等待

■ WaitForSingleObject

- 在指定的时间内等待指定对象为有信号状态(signaled state)

```
DWORD WaitForSingleObject(  
    HANDLE hHandle,          // handle of object to wait for  
    DWORD dwMilliseconds    // time-out interval in milliseconds  
);
```

同步对象等待

■ WaitForMultipleObjects

- 在指定的时间内等待多个对象为有信号状态

```
DWORD WaitForMultipleObjects(  
    DWORD nCount,                //The number of object handles in the array  
                                //pointed to by lpHandles  
    CONST HANDLE *lpHandles,    // An array of object handles  
    BOOL bWaitAll,               // TRUE, the function returns when the state  
                                //of all objects in the lpHandles array is signaled  
    DWORD dwMilliseconds        // time-out interval in milliseconds  
);
```

临界区对象(Critical Section)

- 不是内核支持的同步对象，只能在同一进程内使用
- 同一进程内各线程对临界区对象的访问是互斥进行的
- 把变量说明为CRITICAL_SECTION类型，就可作为临界区使用

临界区对象(Critical Section)

■ 临界区对象有关的API:

- InitializeCriticalSection对临界区对象进行初始化
- EnterCriticalSection等待占用临界区的使用权，得到使用权时返回
- TryEnterCriticalSection非等待方式申请临界区的使用权；申请失败时，返回0
- LeaveCriticalSection释放临界区的使用权
- DeleteCriticalSection释放与临界区对象相关的所有系统资源

生产者-消费者问题

- HANDLE hFull; /*满信号量*/
- HANDLE hEmpty; /*空信号量*/
- HANDLE hMutex; /*互斥量: 保护缓冲区互斥访问*/

Producer

P(empty);
P(mutex); //进入区
one unit --> buffer;
V(mutex);
V(full); //退出区

Consumer

P(full);
P(mutex); //进入区
one unit <-- buffer;
V(mutex);
V(empty); //退出区

生产者-消费者问题

```
DWORD WINAPI producer(LPVOID lpParam)
{
    int id = *(int *)lpParam;
    LARGE_INTEGER largeInt;
    QueryPerformanceCounter(&largeInt);
    int product_id = largeInt.LowPart % 1000;
    printf("Producer %d started\n", id);
    WaitForSingleObject(hEmpty, INFINITE); //获取hEmpty信号量
    WaitForSingleObject(hMutex, INFINITE); //获取互斥量

    printf("Producer #%d is producing product: %d\n", id, product_id);
    buffer[in] = product_id;
    in = (in + 1) % BUFFSIZE;

    ReleaseMutex(hMutex); //释放互斥量
    ReleaseSemaphore(hFull, 1, NULL); //释放hFull信号量

    printf("Producer %d ended\n", id);
    ExitThread(0);
}
```

生产者-消费者问题

```
DWORD WINAPI consumer(LPVOID lpParam)
{
    int id = *(int *)lpParam;
    int product_id;
    printf("Consumer %d started\n", id);
    WaitForSingleObject(hFull, INFINITE); //获取hFull信号量
    WaitForSingleObject(hMutex, INFINITE); //获取互斥量
    product_id = buffer[out];
    out = (out + 1) % BUFFSIZE;
    printf("Consumer #%d is consuming product: %d\n", id, product_id);

    ReleaseMutex(hMutex); //解锁互斥量
    ReleaseSemaphore(hEmpty, 1, NULL); //释放hEmpty信号量

    Sleep(1000);
    printf("Consumer %d ended\n", id);
    ExitThread(0);
}
```


生产者-消费者问题

```
int main()
{
    .....
    hFull = CreateSemaphore(NULL, 0, BUFFSIZE, NULL);
    hEmpty = CreateSemaphore(NULL, BUFFSIZE, BUFFSIZE, NULL);
    hMutex = CreateMutex(NULL, FALSE, NULL);

    //创建生产者
    for (i = 0; i < N_PRODUCER; ++i)
        thread[i] = CreateThread(NULL, 0, producer, (void*)&i, 0, NULL);
    //创建消费者
    for (i = 0; i < N_CONSUMER; ++i)
        thread[N_PRODUCER + i] = CreateThread(NULL, 0, consumer, (void*)&i, 0, NULL);

    //等待线程结束
    WaitForMultipleObjects(N_PRODUCER + N_CONSUMER, thread, TRUE, INFINITE);
    CloseHandle(hMutex);
    CloseHandle(hEmpty);
    CloseHandle(hFull);
}
```

大纲

01

Linux的互斥和同步机制

02

Windows的互斥和同步机制

03

进程间高级通信

进程间高级通信

- 信号量、互斥锁和条件变量等同步与互斥机制的核心作用是建立进程（或线程）之间的时序约束，从而保证对共享资源的有序访问并防止竞争条件（race condition）
- 这些机制主要用于协调执行顺序和传递状态信息，虽然可以承载少量控制信息，但其表达能力有限，不适合用于复杂或大规模数据的传输
- 因此，需要在进程之间进行结构化或大量数据交换时，通常需要引入更高层次的通信机制

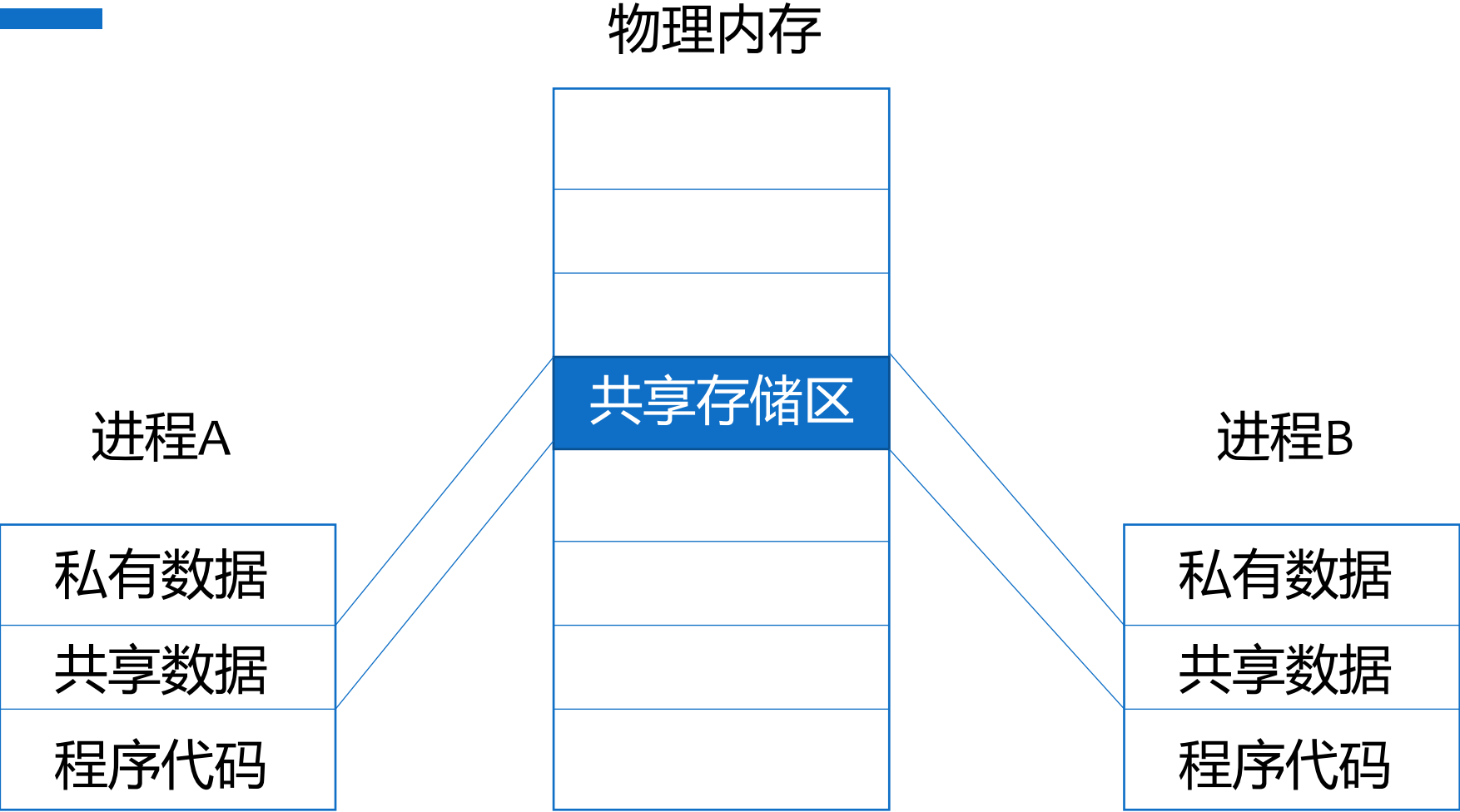
高级通信的常用机制

- 进程间高级通信用于实现进程之间的信息共享和信息交换。常用的进程间高级通信方式包括
 - 共享内存
 - 消息传递
 - 管道
 - 套接字(socket)
- 管道、消息队列、socket 在常规实现中通常需要在进程地址空间之间复制数据，而共享内存通过映射同一物理页避免了这类复制。在现代操作系统中，前三者一般通过零拷贝（zero-copy）技术减少或者避免数据复制

共享内存

- 共享内存是指在物理内存中划出一块共享存储区，相互通信的进程可以将共享存储区映射到各自的进程地址空间，通过读或写共享存储区数据实现通信
- 两个不同进程A、B将同一块物理内存被映射到各自的进程地址空间，进程A可以即时看到进程B对共享内存中数据的更新，反之亦然

共享内存



共享内存

- 共享内存通信需要解决两个问题：
 - 怎样提供共享内存
 - 共享内存的读写互斥问题
- 操作系统一般只提供共享的内存空间
- 处理进程间的读写互斥问题由程序开发人员负责解决，通过进程的互斥与同步来保证数据一致性

Linux共享内存机制

- Linux有三种共享内存实现机制
 - 系统调用mmap()
 - POSIX共享内存
 - System V 共享内存

系统调用mmap()

- mmap函数把一个文件或一个POSIX共享内存区对象映射到调用进程的地址空间。使用该函数有三个目的：
 1. 使用普通文件以提供内存映射I/O
 2. 使用特殊文件以提供匿名内存映射
 3. 使用shm_open以提供无亲缘关系进程间的POSIX共享内存区

系统调用mmap()

- mmap()——把文件映射到一个存储区域中

- 函数原型

```
#include <sys/mman.h>
```

```
void *mmap(void *addr, size_t len, int prot, int flag, int filedес, off_t off);
```

- 参数:

- addr——指向映射存储区的起始。通常设置为NULL，表示由系统选择该映射区的起始地址

- len——映射的字节

- prot——对映射存储区的保护要求(PROT_READ、 PROT_WRITE、 PROT_EXEC)

系统调用mmap()

- mmap参数(续):
 - flag——flag标志位(MAP_SHARED、MAP_PRIVATE、MAP_ANONYMOUS)
 - filesdes——要被映射文件的描述符, 在映射该文件到一个地址空间之前, 先要打开该文件
 - off——要映射字节在文件中的起始偏移量, 通常将其设置为0
- 返回值: 若成功则返回映射区的起始地址, 若出错则返回MAP_FAILED

系统调用mmap()

■ munmap()——解除存储映射

■ 函数原型

```
#include <sys/mman.h>
```

```
int munmap(caddr_t addr, size_t len);
```

■ 参数:

■ addr——指向映射存储区的起始地址，其中addr参数是由mmap返回的地址

■ len——映射的字节

■ 返回值：若成功则返回0，若出错则返回-1

系统调用mmap()

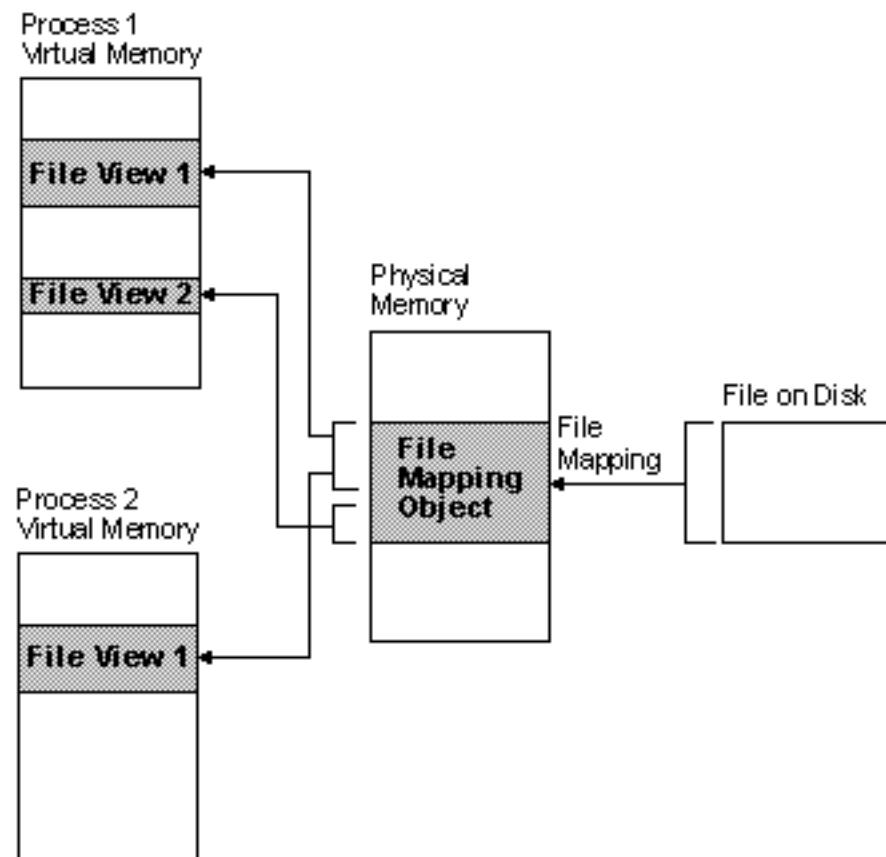
■ 示例：创建4字节大小的匿名共享内存区域，实现通信

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/mman.h>
#include <sys/wait.h>
#define N 10

int main()
{
    int i, sum;
    int *result_ptr = mmap(0, 4, PROT_READ|PROT_WRITE, MAP_SHARED|MAP_ANONYMOUS, 0, 0);
    int pid=fork();
    if(pid==0)
    {
        for(sum=0, i=1; i<N; i++) sum += i;
        *result_ptr = sum;
    }
    else
    {
        wait(0);
        printf("result = %d\n", *result_ptr);
    }
    munmap(result_ptr, 4);
}
```

Windows的共享内存

- Windows 采用文件映射机制实现共享存储区
- 采用文件映射机制，用户进程可以将整个文件映射为进程虚拟地址空间的一部分来加以访问

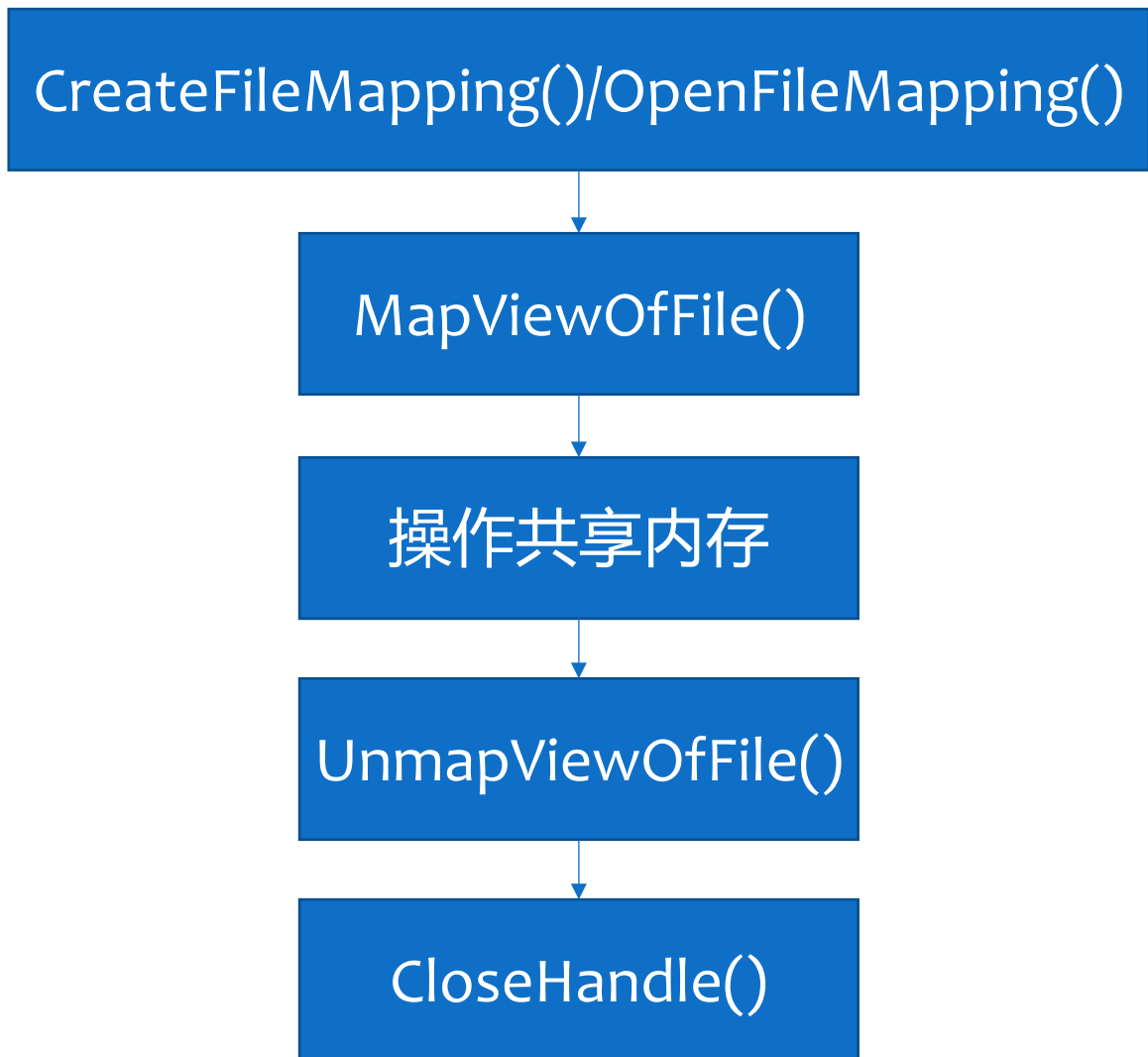


Windows的共享内存

■与共享内存相关的API

- CreateFileMapping——为指定文件创建或打开一个文件映射对象，返回对象句柄
- OpenFileMapping——打开一个命名的文件映射对象，返回对象句柄
 - 使用CreateFileMapping和OpenFileMapping时可以指定对象名称
- MapViewOfFile——将文件视图映射到本进程的地址空间，返回映射地址空间的首地址，之后可利用首地址进行读写
- UnmapViewOfFile——解除文件映射与本进程地址空间间映射关系
- CloseHandle——关闭文件映射对象

Windows的共享内存



Windows的共享内存

■实例



common.h



SHM_Writer.c



SHM_Reader.c

共享内存

- **优点：**效率高，因为进程可以直接读写内存，而不需要任何数据的拷贝
- **缺点：**
 - 进程必须在同一台物理机器上才能使用内存
 - 安全性脆弱
- 使用全局变量在同一个进程内部的线程之间实现通信不称为共享内存

消息传递

- 在消息传递系统中，进程间的数据交换是以消息(message, 在计算机网络中又称报文)为单位
- 程序员直接利用系统提供的一组通讯命令（原语）来实现通讯
- 因其实现方法的不同，消息传递可分为直接通信和间接通信
- 在直接通信中，发送进程和接收进程需要显式指定对方标识（如进程ID），实现点对点通信
- 在间接通信中，进程通过中介实体（如邮箱或消息队列）进行通信，发送者和接收者之间无需直接建立关联

消息传递——直接通信方式

- 发送和接收进程都必须以显式方式提供目标进程标识符，以表明向谁发送或者从谁那里接收消息
- 系统提供的通信原语：
send(receiver, message)
receive(sender, message)

用消息传递实现生产者-消费者问题

```
#define N 100                                /* number of slots in the buffer */

void producer(void)
{
    int item;
    message m;                                /* message buffer */

    while (TRUE) {
        item = produce_item();               /* generate something to put in buffer */
        receive(consumer, &m);               /* wait for an empty to arrive */
        build_message(&m, item);             /* construct a message to send */
        send(consumer, &m);                  /* send item to consumer */
    }
}

void consumer(void)
{
    int item, i;
    message m;

    for (i = 0; i < N; i++) send(producer, &m); /* send N empties */
    while (TRUE) {
        receive(producer, &m);               /* get message containing item */
        item = extract_item(&m);             /* extract item from message */
        send(producer, &m);                 /* send back empty reply */
        consume_item(item);                 /* do something with the item */
    }
}
```

直接通信的设计问题

- 直接通信主要存在于早期微内核系统中，在现代操作系统中逐渐被淘汰。
直接通信的主要问题：
 - 通信关系绑定在进程身份上，收发双方必须知道对方。在执行`send(receiver, message)`时，如果`receiver`崩溃，则通信失败；如果`receiver`重启，PID可能变化
 - 现代系统真正需要的是把消息发给某个服务，例如文件系统服务、网络服务，而不是某个进程。`send(fs_server, message)`的问题在于隐含假设文件系统服务 = 某个固定进程。而服务要求可以重启，可以迁移
 - 无法表达复杂通信关系。`send(receiver, message)`只能实现一对一通信，无法实现多对一（服务）和一对多（广播）

消息传递——间接通信方式

- 在消息传递机制中，间接通信的中介主要是各种逻辑消息对象，典型包括：
- 邮箱 (mailbox)
- 消息队列 (message queue)

信箱

- 利用信箱通讯方式，发送者不必写出接收者，可以向不知名的进程发送消息，由系统选择接收者
- 由于消息可以完好的保存在信箱里，只允许核准的目标用户读取，因此利用这个方法可以实现实时通信也可以实现非实时通信



信箱

■ 消息的发送和接收

`send(mailbox, message)`

`receive(mailbox, message)`

Windows的邮件槽

- Windows中的邮件槽(mailslot)是信箱通讯方式的一种具体实现，通常采用client-server模式
- 单向：从client发往server。server负责创建邮件槽，它可从邮件槽中读消息；client可利用邮件槽的名字向它发送消息
- 邮件槽名字：
 - 作为 client(发送方) 打开邮件槽时，可以是“\\range\\mailslot\\[path]name”，这里range可以是 .(local computer), computerName, domainName, *(primary domain)
 - 作为 server(接收方) 只能在本机建立邮件槽，名字可为“\\.\mailslot\\[path]name”

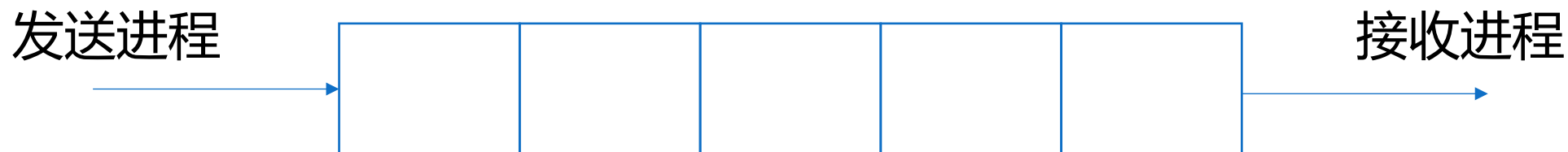
Windows的邮件槽

■与邮件槽相关的API

- CreateMailslot——服务器方创建邮件槽，返回其句柄
- GetMailslotInfo——服务器查询邮件槽的信息，如：消息长度、消息数目、读操作等待时限等
- SetMailslotInfo——服务器设置读操作等待时限
- ReadFile——服务器读邮件槽
- CreateFile——客户方打开邮件槽
- WriteFile——客户方发送消息

消息队列

- 消息队列是一种由操作系统维护的、按一定规则组织的消息链表（或队列），进程通过它进行间接通信
- 消息队列的操作：
 `send(queue, message)`
 `receive(queue, message)`
- 消息队列实现了发送方和接收方完全解耦，可以支持异步通信



Linux的消息队列

- Linux支持两种消息队列机制
 - POSIX消息队列
 - System V消息队列

System V IPC概述

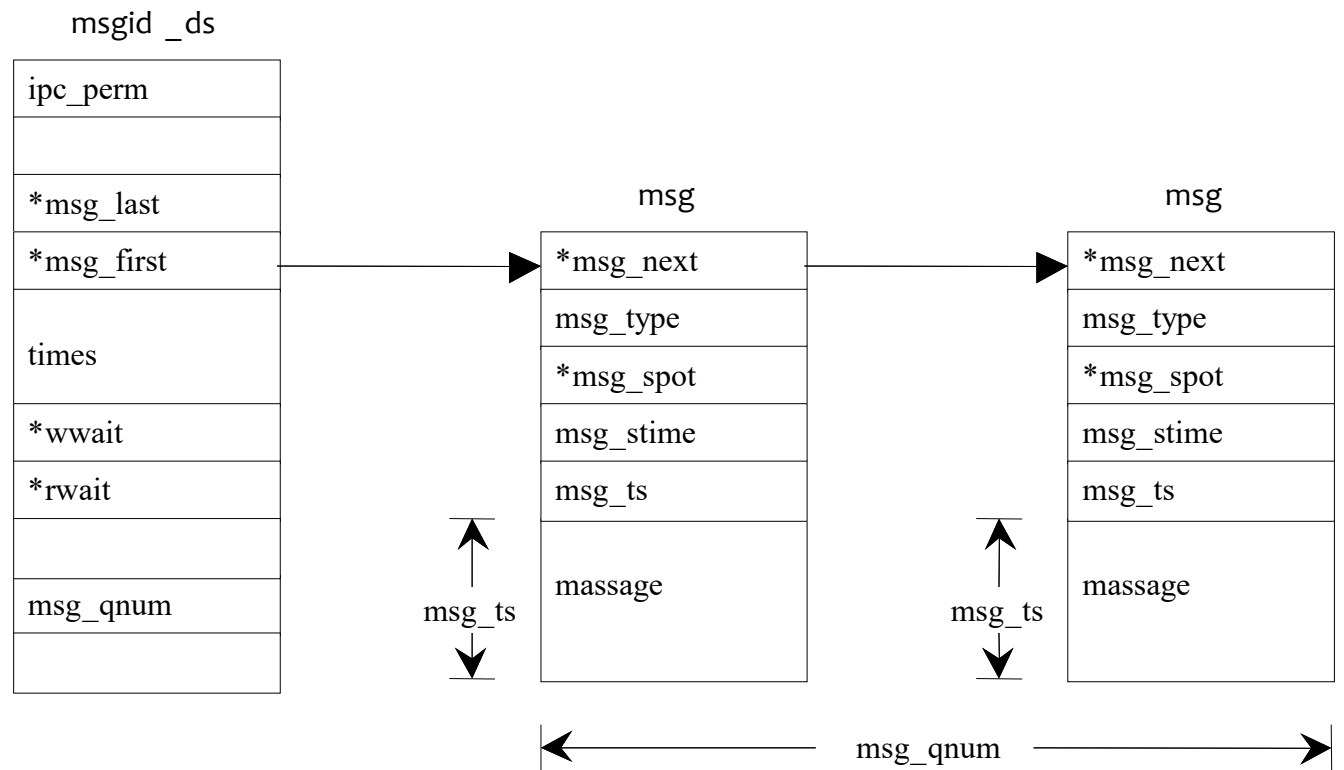
- System V引入了三种高级进程间的通信机制：消息队列、共享内存和信号量
- System V IPC机制主要有如下特点：
- 如果进程要访问System V IPC对象，则需要在系统调用中传递唯一的引用标识符
- 对System V IPC对象的访问，必须经过类似文件访问的许可检验。对这些对象访问权限的设置由对象的创建者利用系统调用设置
- 对象的引用标识符由IPC机制作为访问对象表的索引

System V IPC概述

- 所有表示System V IPC对象的数据结构中都包含一个ipc_perm结构，该结构中包含了作为对象所有者和创建者的进程的用户标识符和组标识符，以及对象的访问模式和对象的访问键
- 对System V IPC对象的访问，必须经过类似文件访问的许可检验。对这些对象访问权限的设置由对象的创建者利用系统调用设置
- 访问键用来定位System V IPC对象的引用标识符。如果进程要访问System V IPC对象，则需要在系统调用中传递唯一的引用标识符

System V消息队列

- Linux 为系统中所有的消息队列维护一个msgque链表，该链表中的每个指针指向一个msgid_ds结构，该结构完整描述一个消息队列。当建立一个消息队列时，系统从内存中分配一个msgid_ds结构并将指针添加到msgque链表



System V消息队列

■消息队列的写入操作和读取操作是类似的，以消息的写入为例，步骤：

1. 当某个进程要写入消息时，该进程的有效uid和gid首先要和ipc_perm中的访问模式进行比较。如果进程不能写入，系统调用返回错误，写操作结束
2. 如果该进程可以向消息队列写入，则消息可以复制到消息队列的末尾。在进行复制之前，必须判断消息队列当前是否已满。消息的具体内容和应用程序有关，由参与通信的进程约定
3. 如果消息队列中当前没有空间容纳消息，则写入进程被添加到该消息队列的写等待队列(如果有某个进程从消息队列中读取了消息，则系统会唤醒写等待队列中的进程)，否则，内核分配一个msg结构，将消息从进程的地址空间中复制到msg结构，然后将msg添加到队列末尾，这时，系统调用成功返回，写操作结束

System V消息队列

■与消息队列相关的系统调用

- msgget——依据用户给出的键值key，创建新消息队列或打开现有消息队列，返回一个消息队列ID
 - msgsnd——发送消息；
 - msgrcv——接收消息，可以指定消息类型；没有消息时，返回-1
 - msgctl——对消息队列进行控制，如删除消息队列
- 消息队列不随创建它的进程的终止而自动撤销，必须用msgctl(msgqid, IPC_RMID, 0)撤销

System V消息队列

snd.c

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <sys/msg.h>
#define MAX_TEXT 512
#define TRUE 1

struct msgbuf {
    long int msgtype;
    char msgtext[MAX_TEXT];
};

int main()
{
    struct msgbuf msgdata;
    int msgid;
    char buffer[MAX_TEXT];

    if((msgid = msgget((key_t)1234, 0666 | IPC_CREAT)) == -1) {
        fprintf(stderr, "msgget failed with error: %d\n", errno);
        exit(EXIT_FAILURE);
    }
}
```

System V消息队列

snd.c

```
printf("msgid = %d\n", msgid);

while(TRUE) {
    printf("Enter message text: ");
    fgets(buffer, MAX_TEXT, stdin);

    msgdata.msgtype = 1;
    strcpy(msgdata.msgtext, buffer);

    if (msgsnd(msgid, (void *)&msgdata, MAX_TEXT, 0) == -1) {
        fprintf(stderr, "msgsnd failed\n");
        exit(EXIT_FAILURE);
    }
    if (strncmp(buffer, "end", 3) == 0) {
        break;
    }
}
exit(EXIT_SUCCESS);
}
```

System V消息队列

rcv.c

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>
#include <errno.h>
#include <unistd.h>
#include <sys/msg.h>
#define MAX_TEXT 512
#define TRUE 1

struct msgbuf {
    long int msgtype;
    char msgtext[MAX_TEXT];
};

int main()
{
    int msgid;
    struct msgbuf msgdata;

    if((msgid = msgget((key_t)1234, 0666 | IPC_CREAT)) == -1) {
        fprintf(stderr, "msgget failed with error: %d\n", errno);
        exit(EXIT_FAILURE);
    }
}
```

System V消息队列

rcv.c

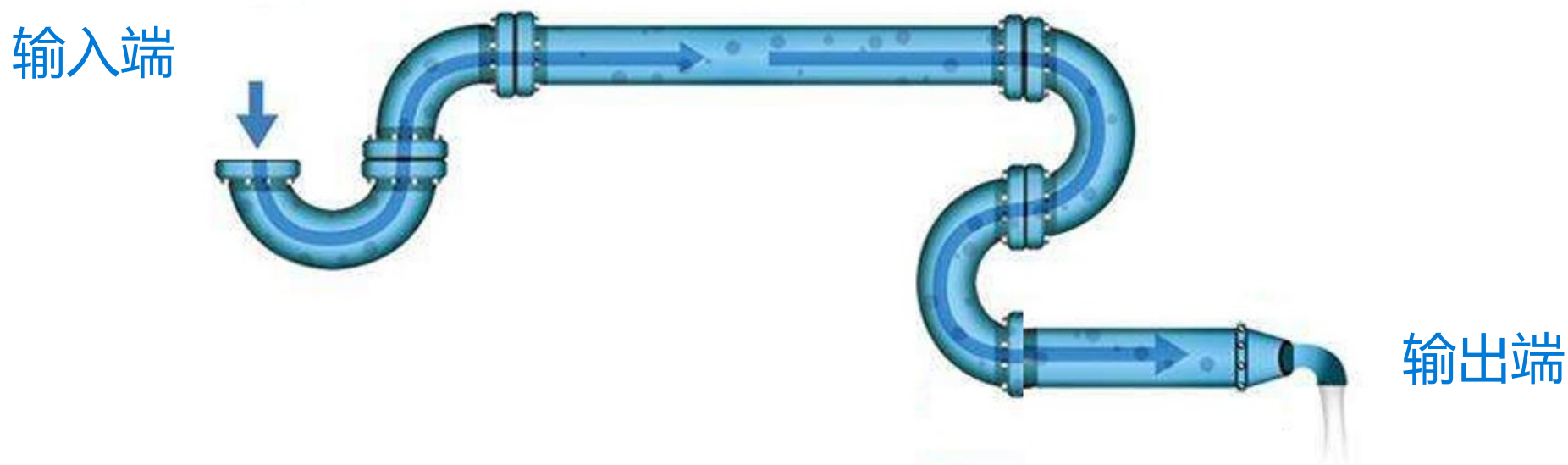
```
printf("msgid = %d\n", msgid);

while(TRUE) {
    if (msgrcv(msgid, (void *)&msgdata, MAX_TEXT, 0, 0) == -1) {
        fprintf(stderr, "msgrcv failed with error: %d\n", errno);
        exit(EXIT_FAILURE);
    }
    printf("Received message: %s", msgdata.msgtext);
    if (strncmp(msgdata.msgtext, "end", 3) == 0) {
        break;
    }
}

if (msgctl(msgid, IPC_RMID, 0) == -1) {
    fprintf(stderr, "msgctl(IPC_RMID) failed\n");
    exit(EXIT_FAILURE);
}
exit(EXIT_SUCCESS);
}
```

管道

- 管道(pipe)是在进程间以字节流方式传送信息的通信通道
- 每个管道具有两个端，一端用于输入，一端用于输出。在相互通信的两个进程中，一个进程将信息送入管道的输入端，另一个进程就可以从管道的输出端读取该信息



管道

- 管道是由UNIX首创的一种借助文件和文件系统形成的一种通信方式。由于其有效性，一些系统继UNIX之后相继引入了管道技术，管道通信将成为进程通信的一种重要方式
- 消息传递通信是以内存缓冲区为基础
- 管道是以文件系统为基础

管道

- 利用管道时，一个进程的输出可成为另外一个进程的输入

- 例：

```
sort < namefile | grep ma
```

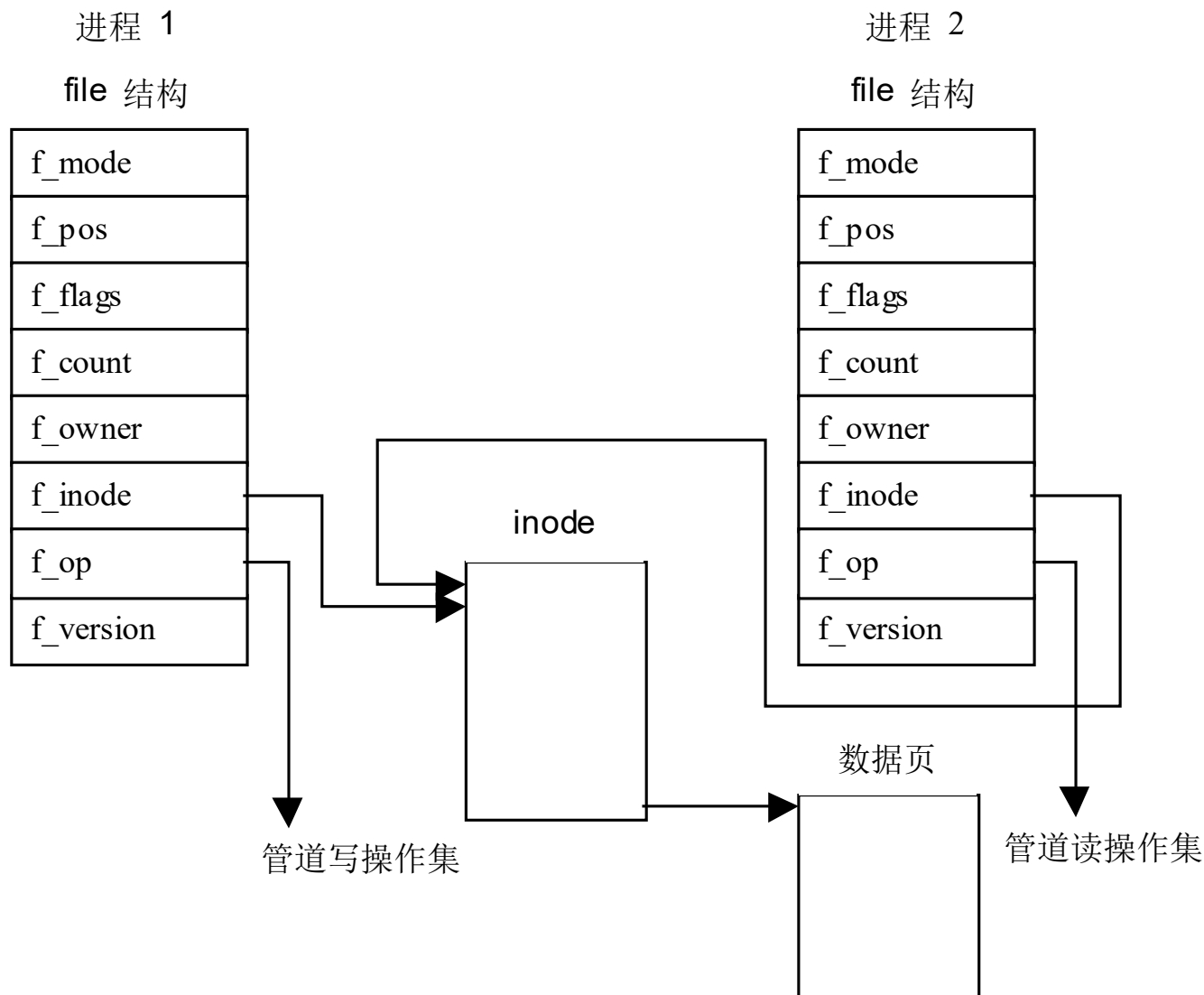
管道

■管道的类型

- 管道(pipe): 也称无名管道(anonymous pipe), 可用于具有亲缘关系的进程间的通信
- 命名管道(named pipe): 有名管道克服了管道没有名字的限制, 因此, 除具有管道所具有的功能外, 它还允许无亲缘关系的进程间的通信

Linux管道

- 在Linux中，通过将两个file结构指向同一个临时的VFS索引节点，而VFS索引节点又指向同一个物理页而实现管道



Linux管道

- 通过pipe系统调用创建无名管道，得到两个文件描述符，分别用于写和读
 - `int pipe(int fildes[2]);`
 - 文件描述符fildes[0]为读端，fildes[1]为写端
 - 通过系统调用write和read进行管道的写和读
- 进程间双向通信，需要两个管道
- 只适用于父子进程之间或父进程安排的各个子进程之间

Linux管道

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>
#include <string.h>

int main(int argc, char *argv[])
{
    int f_des[2];
    static char message[BUFSIZ];
    if(argc!=2){
        fprintf(stderr, "Usage: %s message\n", *argv);
        exit(1);
    }

    if(pipe(f_des)==-1){
        perror("pipe");
        exit(2);
    }
    switch(fork()){
        case -1:
            perror("fork");
            exit(3);
```

```
        case 0:
            close(f_des[1]);
            if(read(f_des[0], message, BUFSIZ)!=-1){
                printf("Message received by child:[%s]\n", message);
                fflush(stdout);
            }else{
                perror("read");
                exit(4);
            }
            break;
        default:
            close(f_des[0]);
            strcpy(message, argv[1]) ;
            if(write(f_des[1], message, BUFSIZ)!=-1){
                printf("Message sent by parent:[%s]\n", argv[1]);
                fflush(stdout);
            }else{
                perror("write");
                exit(5);
            }
    }
}
```

Linux命名管道

- 命名管道也称为FIFO(first-in first-out)，这是因为这种管道的操作方式基于“先进先出”原理。命名管道中，首先写入管道的数据是首先被读出的数据
- 匿名管道是临时对象，而FIFO则是文件系统的真正实体，如果进程有足够的权限就可以使用FIFO
- FIFO和匿名管道的数据结构以及操作极其类似，二者的主要区别在于，FIFO在使用之前就已存在，用户可打开或关闭FIFO；而匿名管道在只在操作时存在，是临时对象

Linux命名管道

- 命名管道可通过mknod系统调用建立：指定mode为S_IFIFO
`int mknod(const char *path, mode_t mode, dev_t dev);`

Windows管道

- Windows提供了匿名管道和命名管道两种管道机制
- 利用CreatePipe可创建匿名管道，得到两个读写句柄；利用ReadFile和WriteFile可进行匿名管道的读写

```
BOOL CreatePipe( PHANDLE hReadPipe,      // address of variable for read handle
                 PHANDLE hWritePipe,     // address of variable for write handle
                 LPSECURITY_ATTRIBUTES lpPipeAttributes, // pointer to security attributes
                 DWORD nSize              // number of bytes reserved for pipe
                );
```


Windows管道

```
#include <windows.h>
#include <stdio.h>
#define BUFSIZE 1024
```

```
main()
{
    SECURITY_ATTRIBUTES sa;
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    HANDLE hRead, hWrite;
    DWORD bytesRead;
    char buffer[BUFSIZE] = {0};

    sa.nLength = sizeof(SECURITY_ATTRIBUTES);
    sa.lpSecurityDescriptor = NULL;
    sa.bInheritHandle = TRUE;
    if(!CreatePipe(&hRead, &hWrite, &sa, 0))
    {
        printf("Error On CreatePipe()\n");
        return;
    }
}
```

```
si.cb = sizeof(STARTUPINFO);
GetStartupInfo(&si);
si.hStdError = hWrite;
si.hStdOutput = hWrite;
si.wShowWindow = SW_HIDE;
si.dwFlags = STARTF_USESHOWWINDOW | STARTF_USESTDHANDLES;
if (!CreateProcess(NULL, "C:\\\\Windows\\System32\\cmd.exe /C dir",
                  NULL, NULL, TRUE, 0, NULL, NULL, &si, &pi))
{
    printf("Error on CreateProcess()\n");
    return;
}
CloseHandle(hWrite);

while(TRUE)
{
    if (ReadFile(hRead, buffer, BUFSIZE, &bytesRead, NULL) == 0)
        break;

    printf(buffer);
    Sleep(1000);
}
}
```

Windows管道

- **命名管道**：一个服务器端与一个客户进程间的通信通道；可用于不同机器上进程通信
- 通常采用client-server模式连接本机或网络中的两个进程
- **管道名字**
 - 作为客户方(连接到一个命名管道实例的一方)时，可以是"`\\serverName\pipe\pipename`"
 - 作为服务器方(创建命名管道的一方)时，只能取serverName为"`\\.pipe\PipeName`"，不能在其它机器上创建管道

Windows管道

- 与命名管道相关的API
- CreateNamedPipe——在服务器端创建并返回一个命名管道句柄
- ConnectNamedPipe——在服务器端等待客户进程的请求
- CallNamedPipe——从管道客户进程建立与服务器的管道连接
- ReadFile、WriteFile(用于阻塞方式)、ReadFileEx、WriteFileEx(用于非阻塞方式)——用于命名管道的读写

操作系统，2026

谢谢

OPERATING SYSTEM